

Evolución Arquitectónica: Cambios del Modelo de Datos (V1 a V2)

Este documento detalla las mejoras implementadas en el Diagrama Entidad-Relación (DER) de IntellOps, pasando de un prototipo básico (V1) a una arquitectura escalable y lista para producción (V2).

Las modificaciones se agrupan en cuatro grandes áreas de mejora:

1. Soporte Multi-Proyecto (Escalabilidad)

El cambio más grande de la V2 es preparar el sistema para que no se limite a una sola página web.

- **Nueva Tabla APPLICATION:** En la V1, asumíamos que el sistema solo monitoreaba una web. En la V2 agregamos esta tabla maestra. Ahora, IntellOps puede monitorear el "Portal GIDAS", un "Dashboard Admin" y una "Web Pública" al mismo tiempo, sin mezclar sus datos.
- **Modificación en USER_SESSION:** Se agregó el campo app_id (Foreign Key). Ahora cada visita de un usuario está atada a un proyecto específico. Además, se sumaron end_timestamp y duration_ms para saber exactamente cuánto tiempo estuvo la persona navegando.

2. Enriquecimiento de Métricas y Contexto (Mejoras en RUM)

Para que el Machine Learning funcione mejor, necesitamos darle datos más ricos, no solo números fríos.

- **Mejoras en RUM_METRIC:**
 - Pasó de ser una tabla muy básica a una súper completa. Se agregó unit (para saber si el valor son milisegundos o un conteo de clics).
 - Se sumó page_url para saber en qué pantalla exacta ocurrió la lentitud.
 - **Campo estrella (metadata en JSONB):** Se agregó para guardar información extra y flexible (como el ID del botón que falló o el tipo de red del usuario) sin tener que alterar la estructura de la base de datos a cada rato.
- **Cruce de Errores (JS_EXCEPTION):** En la V2, a los errores de JavaScript se les agregó un metric_id opcional. Esto permite saber si un error en la consola fue lo que causó directamente que una métrica (como un clic) fuera anómala.

3. Madurez en la Inteligencia Artificial (MLOps)

El seguimiento de los modelos de ML debe ser profesional y auditable.

- **Mejoras en ML_MODEL:**
 - El `model_id` cambió de ser un número secuencial (int) a un UUID. Esto es clave si en el futuro se entrenan modelos en una computadora distinta y se quieren importar a la base de datos sin que los IDs choquen.
 - Se sumó el campo `status` (para saber si un modelo está "activo", "entrenando" o "deprecado").
 - Se agregó `hyperparameters` (JSONB) para guardar con qué configuración matemática exacta se entrenó ese algoritmo (vital para la investigación científica).
- **Mejoras en ANOMALY:** Se añadió `confidence_score`. No solo guardamos que hubo una anomalía, sino "qué tan seguro" está el algoritmo de que es real (ej. 98% de confianza), lo que ayuda a evitar falsas alarmas.

4. Gestión de Usuarios y Alertas Claras

Mejoras orientadas a la administración del sistema y la comunicación.

- **Mejoras en LAB_USER:** Se agregó `is_active` para poder dar de baja a un administrador sin borrar su historial (borrado lógico), y `last_login` por cuestiones de seguridad y auditoría.
- **Renombre de Tablas:** `USER_FAVORITE` pasó a llamarse `USER_FAVORITE_METRIC` para que su propósito sea inconfundible.
- **Mejoras en ALERT:** Se sumaron los campos `recipient` (a quién se le mandó) y `message` (el texto exacto que se le envió por WhatsApp o Email), logrando así un registro histórico completo de las notificaciones.